

Reinforcement Learning for Compiled-CNOT-Efficient VQE Circuits

Extended Abstract

Josh Joseph John Carlson (CS224R, Spring 2026)

Motivation. The variational quantum eigensolver (VQE) estimates a molecule’s ground-state energy with a parameterized circuit. On near-term hardware the two-qubit CNOT gate is the expensive part, since it dominates both error and runtime. What really matters for an ansatz is therefore its compiled CNOT count, measured after the circuit is decomposed to the native gate set, rather than its number of chemistry excitations. The gap is large in practice. A single double-excitation rotation compiles to fourteen CNOTs, and an accurate chemistry ansatz stacks several of them. We ask whether reinforcement learning (RL) can design circuits that reach a fixed accuracy target with fewer compiled CNOTs than strong hand-designed and greedy baselines.

Method. We treat circuit design as a bilevel problem. An RL agent trained with proximal policy optimization (PPO) picks the circuit structure in an outer loop, while a classical optimizer (L-BFGS-B with adjoint gradients) fits the gate angles in an inner loop and returns the energy. We try three pipelines. One builds a circuit from raw gates, one runs ADAPT-VQE and then compiles and prunes, and one prunes an already compiled circuit with RL. Every comparison uses a matched-start protocol. The agent and a greedy backward-elimination baseline begin from the identical compiled circuit, share the same inner optimizer, and are scored by the same compiled-CNOT metric.

Implementation. The agent is a masked actor-critic with a value baseline. Training uses Monte-Carlo returns in the terminal-reward settings, advantage normalization, an entropy bonus, and a behavioral-cloning warm start from the greedy removal order. We build the molecules (H_2 and LiH active spaces of six, eight, and ten qubits) in PennyLane. The larger LiH runs were trained on GPUs through Modal, where the inner VQE solve is the dominant cost.

Results. On H_2 , raw-gate RL finds a three-CNOT circuit that reproduces the exact ground state, against fourteen for one excitation and seven for compile-then-prune. On LiH (eight qubits) at chemical accuracy (1.6 mHa), RL matches greedy at the seven-CNOT floor from a minimal double-excitation start across four runs. From a full ADAPT start it reaches eleven to twenty CNOTs, where greedy stalls at twenty-three. At exact FCI the two methods tie at twenty-five CNOTs.

Discussion. There are two kinds of improvement here, and they are easy to confuse. One is the global accuracy floor, where greedy and RL tie. The other is same-start improvement, where the learned non-greedy search actually helps. Greedy backward elimination is a strong baseline, and the value of RL is in escaping the greedy frontier on deep starting circuits rather than in finding a smaller global optimum.

Conclusion. RL is a usable tool for circuit compression. It solves the small H_2 case, gives a genuine same-start improvement on LiH , and matches greedy everywhere else. Scaling to ten qubits runs into the computational limits of both methods, and that is the main direction for future work.

Reinforcement Learning for Compiled-CNOT-Efficient Variational Quantum Eigensolver Circuits

Josh Joseph
Stanford University
joshjo@stanford.edu

John Carlson
Stanford University
jwc10@stanford.edu

Abstract

We train proximal policy optimization (PPO) agents to minimize the compiled CNOT count of variational quantum eigensolver (VQE) circuits for H_2 and LiH at a fixed energy accuracy. Prior RL-for-VQE work usually benchmarks against the deep UCCSD ansatz. We instead compare against ADAPT-VQE and greedy backward elimination, using a matched-start protocol in which the agent and the baseline begin from the identical compiled circuit and share the same inner optimizer. On H_2 , raw-gate RL from scratch finds a three-CNOT circuit at the exact ground-state energy. On the eight-qubit LiH active space, RL matches greedy at the seven-CNOT chemical-accuracy floor from a minimal start across four runs, and reaches eleven to twenty CNOTs from a full ADAPT start where greedy reaches twenty-three. The two methods tie at twenty-five CNOTs at exact FCI. We report results across multiple seeds, analyze why these outcomes differ, and show that a wide range of CNOT counts share the same loose chemical-accuracy energy.

1 Introduction

Estimating the ground-state energy of a molecule is a central problem in quantum chemistry. It determines bonding, reaction barriers, and binding affinities, and its exact classical solution scales exponentially in the number of orbitals. The variational quantum eigensolver (VQE) is a leading near-term quantum approach Peruzzo et al. [2014], McClean et al. [2016]. A parameterized circuit $U(\theta)$ prepares a trial state $|\psi(\theta)\rangle$, the energy $E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle$ is measured, and a classical optimizer minimizes it. The variational principle guarantees $E(\theta) \geq E_0$, so any trial energy is an upper bound on the true ground state.

On noisy intermediate-scale quantum (NISQ) hardware Preskill [2018], the binding constraint is circuit depth, and in particular the count of two-qubit CNOT gates. These gates are roughly an order of magnitude noisier and slower than single-qubit rotations, and they dominate the error budget. The cost that matters is therefore the number of CNOTs after the circuit is compiled to a native gate set such as $\{R_X, R_Y, R_Z, \text{CNOT}\}$. This distinction is large in practice. A single fermionic double-excitation rotation, the workhorse of chemistry ansatzes, compiles to fourteen CNOTs, and a useful ansatz stacks several. Reducing the excitation count is not the same as reducing the compiled CNOT count.

The question we ask is a simple one. Can a reinforcement-learning agent design VQE circuits that reach a fixed accuracy target with fewer compiled CNOTs than strong classical baselines? Throughout, we treat the compiled CNOT count at a fixed accuracy, either chemical accuracy at 1.6 mHa or near-exact FCI, as the single figure of merit. We make three contributions. The first is a bilevel RL formulation that searches circuit structure with PPO while a classical optimizer fits the angles.

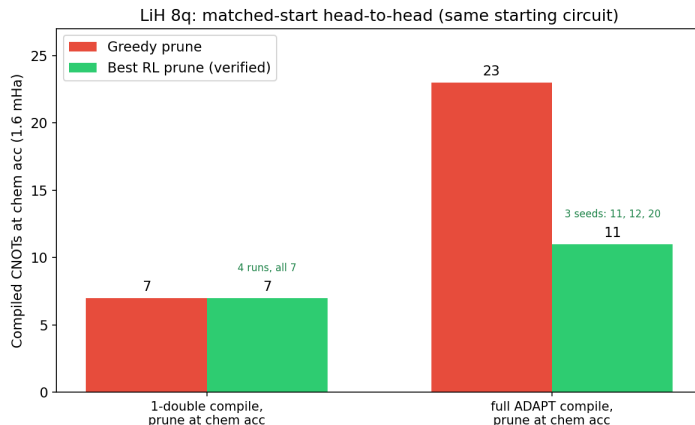


Figure 1: Matched-start comparison on LiH (eight qubits) at chemical accuracy. Each pair fixes the same starting circuit before pruning. From a one-double start both methods reach the seven-CNOT floor. From a full ADAPT start, greedy reaches twenty-three CNOTs while RL prunes to eleven, the best of three completed runs that span eleven to twenty CNOTs (Section 5).

The second is a matched-start evaluation protocol that compares RL against greedy pruning from the identical starting circuit. The third is a multi-scale empirical study on H_2 and three LiH active spaces, reported across seeds, that tells a genuine RL improvement apart from the cases where RL only matches a strong baseline.

2 Related Work

VQE and ansatzes. The unitary coupled-cluster singles-and-doubles (UCCSD) ansatz is chemically motivated but deep, and costs hundreds of CNOTs even for small molecules. Hardware-efficient ansatzes Kandala et al. [2017] are shallower but can suffer trainability problems. ADAPT-VQE Grimsley et al. [2019] grows an ansatz greedily, adding at each step the operator with the largest energy gradient, and reaches chemical accuracy with far fewer operators than UCCSD. Qubit-ADAPT Tang et al. [2021] reduces the cost further. ADAPT is a stronger baseline than UCCSD, and it is the one we adopt.

RL for quantum architecture search. Ostaszewski et al. [2021] frame ansatz construction as an RL problem with a dense, energy-based reward and show that RL can find compact circuits. Later curriculum-based approaches refine the reward schedule Kundu et al. [2024]. Most of this work benchmarks against UCCSD and reports energy or gate count in general terms. We depart from it in two ways. We optimize and report compiled CNOTs at a fixed accuracy, and we benchmark against ADAPT and greedy backward elimination under a matched-start protocol, which is a harder bar than beating UCCSD.

RL algorithm. We use PPO Schulman et al. [2017] for its stable clipped updates, and we compare it against a REINFORCE Williams [1992] baseline in the excitation-level setting. Circuits are simulated and differentiated with PennyLane Bergholm et al. [2018].

3 Method

3.1 Problem formulation

We separate structure from angles. The RL agent chooses the discrete structure of the circuit in an outer loop. Given a structure, a classical optimizer fits the continuous gate angles in an inner loop and returns the optimized energy. The inner solver is L-BFGS-B with PennyLane adjoint gradients and angle warm starting. It is identical across all methods, so that comparisons reflect structure search rather than differences in the angle optimizer. After the agent fixes a structure, we compile

it to $\{R_X, R_Y, R_Z, \text{CNOT}\}$ and count CNOTs. We use two accuracy targets: chemical accuracy, $|E - E_{\text{FCI}}| < 1.6 \text{ mHa}$, and exact FCI, $|E - E_{\text{FCI}}| < 10^{-6} \text{ Ha}$, which is tighter and admits fewer valid pruned circuits.

3.2 Three pipelines

We study three ways to produce a circuit. In raw-gate construction from scratch, the agent places R_X , R_Y , R_Z , and CNOT gates, and optionally structured Givens rotations, onto an initially empty circuit. In the ADAPT-then-prune pipeline, a strong chemistry circuit is built by ADAPT, compiled, and then shortened. In RL pruning, the agent removes gates from a compiled circuit one at a time. The first pipeline is the most general, and the third is the source of our main LiH results.

3.3 Reinforcement learning approach

Each pipeline defines a Markov decision process. The state encodes the current circuit, an action adds or removes a gate or stops the episode, and the reward measures progress toward the accuracy target at a controlled gate cost. Episodes are short, on the order of the circuit length, which shapes several of our design choices below.

Policy and value network. The agent is an actor-critic with a shared backbone of two fully connected layers with Tanh activations, followed by a policy head over the discrete action set and a scalar value head. We keep the network small on purpose. The state is a compact fixed-length feature vector rather than raw circuit text, so a wider or deeper model is unnecessary and would only slow the already expensive inner-loop evaluations. Invalid actions, such as removing an already removed gate or adding a duplicate excitation, are masked by setting their logits to a large negative value before the softmax. Masking keeps the policy distribution over legal moves only and avoids wasting probability mass on infeasible actions.

Algorithm choice. We train the policy with PPO. We chose it over plain policy gradients for a few reasons. The clipped surrogate objective limits how far each update can move, which keeps a single noisy batch from collapsing the policy. The learned value baseline and per-batch advantage normalization cut the gradient variance. Reusing each batch for several epochs also makes better use of every sample, which matters here because every reward evaluation needs an expensive inner VQE solve. As a point of comparison we implemented REINFORCE with a running-mean return baseline for the excitation-level task, and we found it noisier and slower to converge than PPO at the same budget, which lines up with the variance arguments above.

Returns and advantages. In the structure-search and pruning settings the reward is given at the terminal step, so the return for every step in an episode equals that terminal reward. We therefore use undiscounted Monte-Carlo returns ($\gamma = 1$) rather than a bootstrapped or generalized-advantage estimator, since the short, terminal-reward episodes make a learned multi-step estimator unnecessary. The advantage is the return minus the value baseline, normalized within each batch. For the dense raw-gate pipeline, where rewards arrive at every step, we use discounted returns with γ between 0.95 and 0.97. The objective combines the clipped policy loss, a mean-squared-error value loss with coefficient 0.5, and an entropy bonus.

Exploration and warm starting. We add an entropy regularizer to the objective to maintain exploration and to keep the policy from committing early to a suboptimal removal order. For the pruning agent we warm start the policy by behavioral cloning on the greedy removal trace. We record the sequence of gate removals that greedy backward elimination performs, then pretrain the policy to imitate that sequence with a cross-entropy loss. PPO then fine-tunes from this initialization and is free to discover non-greedy removal orders that greedy cannot reach. This use of imitation to seed an on-policy method is a standard technique, and it both speeds up training and gives the agent a strong starting point relative to the greedy baseline it is measured against.

Reward and curriculum. For the dense pipelines we use the Ostaszewski-style reward Ostaszewski et al. [2021], in which each gate earns the fraction of the remaining correlation energy

Table 1: Main PPO hyperparameters for the LiH pruning agent. The structure-search agent uses a width-128 network with $\gamma = 1$ and entropy coefficient 0.01. The raw-gate agent uses width 256 with $\gamma = 0.95$. The local smoke setting uses width 128, eight episodes per update, and fifteen updates for quick validation.

Hyperparameter	Value
Network	2-layer MLP, Tanh, width 512
Optimizer	Adam, learning rate 3×10^{-3}
PPO clip ϵ	0.2
PPO epochs per update	4
Episodes per update	24
Discount γ	0.97
Entropy coefficient	0.02
Value-loss coefficient	0.5
Updates	100 (one-double), 55 (full ADAPT)
Behavioral-cloning epochs	20 to 50
Order-encoding length k	4
Inner VQE (L-BFGS-B) max iterations	60

it recovers, with a per-CNOT penalty and a terminal bonus:

$$r_t = \frac{E_{t-1} - E_t}{E_{t-1} - E_{\text{FCI}}} - \beta [\text{gate is CNOT}], \quad r_{\text{terminal}} = \pm 5 \text{ (target met or missed)}. \quad (1)$$

The structure-search variant uses a sparser terminal reward equal to the correlation energy recovered minus a depth penalty. On the hardest targets we apply a curriculum: a moving accuracy threshold that starts loose and tightens toward chemical accuracy as the agent succeeds, so the agent receives a useful learning signal before it can satisfy the final target.

State encoding. For the pruning agent the state is a compact, order-aware feature vector. It contains bag-of-gate counts by type, the current depth fraction, the fraction of correlation energy recovered, and a one-hot encoding of the last k gate indices. The order encoding follows the spirit of Ostaszewski et al. [2021] and gives the policy enough recent-history information without the cost of a full sequence model.

Course techniques. The approach draws directly on methods from the course: the actor-critic form of policy gradients, the PPO clipped surrogate objective, advantage estimation with a value baseline and normalization, entropy regularization for exploration, behavioral cloning to warm start from a demonstration, reward shaping, and curriculum learning. Table 1 lists the main hyperparameters.

3.4 Matched-start comparison protocol

Each comparison fixes four things: an identical starting circuit, an accuracy target, a greedy backward-elimination baseline on that start, and an RL agent that prunes from the same start with the same inner optimizer and CNOT metric (Figure 1). Fixing the starting circuit is essential. It rules out the misleading practice of comparing an RL result obtained from one starting circuit against a greedy result obtained from a different and easier one.

4 Experimental Setup

Molecules and active spaces. We use H_2 (two electrons, two orbitals, four qubits) and three LiH active spaces: six qubits (2e, 3o), eight qubits (2e, 4o, our main system), and ten qubits (2e, 5o, a scalability stress test). Hamiltonians are built in PennyLane under the Jordan-Wigner mapping. We note that the LiH systems are active spaces, not the full molecule.

Training and reproducibility. The pruning agent uses the hyperparameters in Table 1. We run a local smoke setting of fifteen updates for validation and longer settings of fifty-five to one hundred

Table 2: H_2 (four qubits) at the exact ground-state energy. The metric is compiled CNOT count.

Method	Compiled CNOTs
ADAPT (one excitation)	14
Compile and prune (greedy or RL)	7
Raw-gate RL from scratch	3

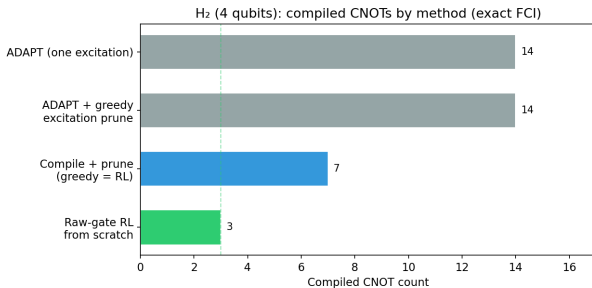


Figure 2: H_2 compiled-CNOT comparison at exact FCI. Raw-gate RL reaches a three-CNOT circuit outside the ADAPT excitation family.

updates on GPUs. Larger runs are dispatched to GPUs through Modal. The dominant cost in every pipeline is the inner VQE solve, which is repeated for every candidate structure, so we cache inner-loop results keyed by structure to avoid recomputation. We repeat the eight-qubit matched-start experiments across multiple random seeds and report the results in Section 5.1.

The chemical-accuracy plateau. On eight-qubit LiH, many distinct compiled circuits, with 7, 12, 14, and 23 CNOTs, share essentially the same energy of about 1.47 mHa above FCI. This is not an artifact. Chemical accuracy is a loose target, and many structures sit on the same correlation plateau. Exact FCI discriminates much more strongly, which is why we report both targets.

5 Results

H_2 . Table 2 and Figure 2 summarize H_2 , where every method reaches the exact ground state, so the comparison reduces to compiled CNOT count. Raw-gate RL from scratch finds a three-CNOT exact circuit, half the seven CNOTs of compile-then-prune and well below the fourteen CNOTs of one excitation. This circuit lies outside the family that ADAPT would propose, because the small effective subspace of H_2 admits a short non-chemistry gate sequence that reaches the ground state. H_2 is small enough that RL solves the combinatorial gate-ordering problem, which validates the pipeline before we scale up.

LiH eight qubits at chemical accuracy. This is our main system. The greedy baselines set the bars. Pruning from a single compiled double excitation reaches a seven-CNOT floor, and pruning from a full ADAPT compile of forty-six CNOTs reaches twenty-three CNOTs, both at the same plateau near 1.47 mHa. Table 3 reports the two matched-start pairs. From the one-double start, RL and greedy both reach seven CNOTs, which is the floor reachable from this start. From the full ADAPT start, RL prunes below the greedy result of twenty-three, with completed runs reaching eleven, twelve, and twenty CNOTs. Figure 3 shows the learning dynamics, in which agents find improvements within a few updates and then hold the best circuit. The contrast between the two starts is the central empirical result of the paper.

5.1 Reproducibility across seeds

Table 4 reports the eight-qubit matched-start results across seeds. From the one-double start the result is fully reproducible. All four runs reach seven CNOTs, so the mean is 7.0 with zero variance, matching the greedy floor of seven. From the full ADAPT start the three completed runs reach

Table 3: LiH (eight qubits) matched-start pairs. Each row fixes the same starting circuit for greedy and RL. Chemical accuracy is 1.6 mHa, and exact FCI is 10^{-6} Ha. RL ranges are over completed runs (Section 5.1).

Start	Target	Greedy CNOTs	RL CNOTs	Outcome
One double	chem. acc.	7	7	Match at floor
Full ADAPT	chem. acc.	23	11 to 20	RL improves on greedy
Full ADAPT	exact FCI	25	25	Match

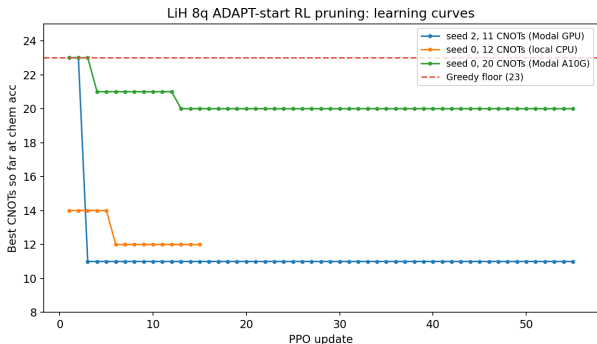


Figure 3: LiH eight-qubit RL pruning from the full ADAPT start across three completed runs. Lower is better. Each run prunes below the greedy result of twenty-three CNOTs within a few PPO updates and then holds the improved circuit.

eleven, twelve, and twenty CNOTs, for a mean of 14.3 CNOTs, a standard deviation of 4.9, and a standard error of 2.8, and a fourth seed reached ten before its job timed out. All four are below the greedy result of twenty-three. We report the ADAPT-start spread with a caveat. These runs used different training budgets and different devices, so part of the variation reflects compute budget rather than seed randomness alone. The one-double comparison is the cleaner reproducibility experiment. The three Modal seeds used an identical preset, and a local smoke run reproduced the same seven-CNOT floor.

LiH eight qubits at exact FCI. At the tighter exact-FCI target from the ADAPT start, greedy and RL both reach twenty-five CNOTs (Table 3). RL does not improve on greedy here. The pruning frontier at exact accuracy is tight, and both methods find the same minimum. We report this as a negative result, because it shows that the RL advantage is specific to the loose, plateau-rich chemical-accuracy regime on a deep starting circuit.

Energy and CNOT tradeoff. Figure 4 plots energy error against compiled CNOTs across the runs. The plateau appears as a near-horizontal band of solutions at about 1.47 mHa that spans a wide range of CNOT counts. This band is the reason the matched-start protocol and the exact-FCI target are both needed to draw meaningful conclusions.

Scaling to six and ten qubits. Figure 5 places the systems on one axis. On six-qubit LiH, greedy pruning from ADAPT reaches sixteen CNOTs in about 514 seconds of CPU time. Partially trained RL runs reached three to five CNOTs, but these were recorded from logs only and not checkpointed to the run volume, so we report them as promising rather than confirmed. On ten-qubit LiH, greedy pruning from the eighty-eight-CNOT ADAPT circuit required about 5.9 hours of CPU time to reach sixty CNOTs, and our GPU RL run timed out before completing. The honest summary at ten qubits is that both methods are computationally strained, which motivates lighter inner solves and warm-started training as future work.

Table 4: LiH eight-qubit pruning across seeds. The one-double Modal seeds share an identical preset, and a local smoke run reproduces the same floor. The full-ADAPT runs varied in training budget and device, so the spread there also reflects budget heterogeneity.

Start	Runs (completed)	Per-run CNOTs	Mean $\pm$ SD	Greedy
One double	4	7, 7, 7, 7	7.0 ± 0.0	7
Full ADAPT	3	11, 12, 20	14.3 ± 4.9	23

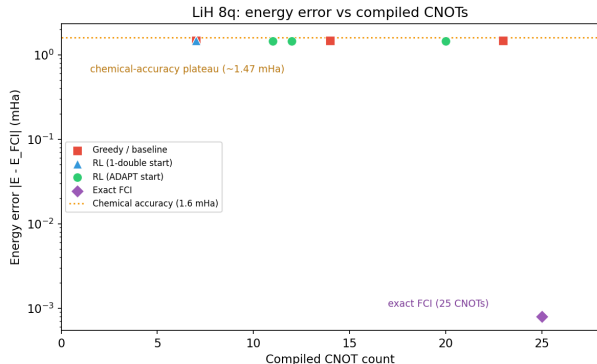


Figure 4: Energy error against compiled CNOTs on LiH eight qubits. The chemical-accuracy plateau is a wide band of solutions at nearly the same energy.

6 Discussion

It helps to separate two kinds of improvement, since they are easy to mix up. The first is the global accuracy floor, the fewest CNOTs any method reaches at a target from the best possible start. On eight-qubit LiH that floor is seven CNOTs at chemical accuracy, reached from a one-double start, and both greedy and RL get there. The second is same-start improvement, where we fix a harder starting circuit and ask whether the agent prunes further than greedy. This is where RL helps, taking the full-ADAPT start from twenty-three CNOTs down to eleven to twenty. Mixing the two up, for example by comparing an RL result from the ADAPT start against a greedy result from the one-double start, would give a misleading claim, and the matched-start protocol is what prevents it.

Greedy backward elimination is a strong baseline because it re-optimizes all angles after each removal, so it rarely overlooks a removable gate near its own frontier. The value of RL is not a smaller global optimum but a learned, non-greedy removal order that escapes local structure which traps greedy on deep ADAPT circuits. The chemical-accuracy plateau amplifies this effect. With many circuits at the same energy, a smarter search has room to find a shorter one. At exact FCI the feasible set is small, and both methods reach the same answer.

7 Limitations

Our study is classical simulation only and does not model hardware noise. It covers a single molecule family, H_2 and LiH active spaces, so we make no claim about cross-molecule transfer. Our training budgets, tens to about one hundred PPO updates, are far below the scale of the original RL-for-VQE work, so the absolute numbers should be read as lower bounds on what RL can achieve. The chemical-accuracy plateau means many CNOT counts share nearly the same energy, so we rely on the exact-FCI target for discrimination. There is a small asymmetry between RL and greedy in inner-optimizer restarts and iteration caps that can shift a greedy result by one CNOT. The six-qubit and ten-qubit RL results are partial and not volume-verified. Finally, our hybrid Givens-then-prune pipeline was not numerically stable enough to support claims and is omitted from the main results.

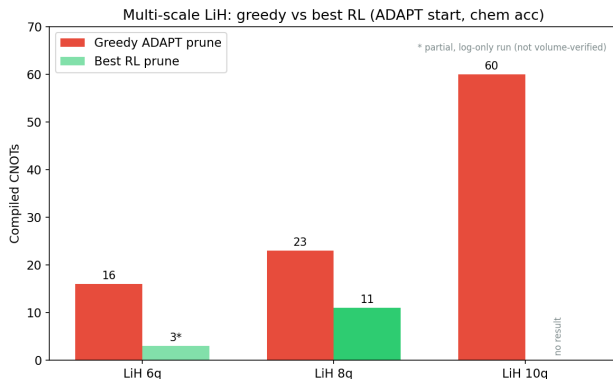


Figure 5: Compiled CNOTs across LiH active spaces. The eight-qubit RL bar is volume-verified. The asterisk marks the six-qubit RL result, which is partial and log-only, and at ten qubits the RL run did not finish, so only the greedy bar is shown.

8 Conclusion

We framed VQE ansatz design as compiled-CNOT minimization at a fixed accuracy and evaluated RL against ADAPT and greedy pruning under a matched-start protocol. RL solves H_2 with a three-CNOT exact circuit, matches greedy at the seven-CNOT LiH chemical-accuracy floor across four runs, and improves on greedy from a full ADAPT start, reaching eleven to twenty CNOTs against twenty-three, while matching greedy at exact FCI. The takeaway is a small but reproducible demonstration that a learned, non-greedy search can compress quantum circuits beyond a strong greedy baseline on deep starting circuits, paired with a protocol that makes such claims trustworthy.

9 Team Contributions

Both of us contributed across the project, and the breakdown below is approximate.

- **Josh Joseph:** the PPO actor-critic, the circuit-structure and pruning environments, the RL pruning trainer with behavioral-cloning warm start, and the GPU training setup. He also wrote the compilation of excitations to native CNOT gates and helped build the greedy pruning baselines.
- **John Carlson:** the VQE inner-loop solver, the ADAPT-VQE baseline, and Hamiltonian and active-space construction. He also implemented the REINFORCE baseline, the reward shaping and accuracy curriculum, and ran the matched-start campaign.

Changes from proposal. We made three substantive adjustments. We pivoted the primary metric from excitation count to compiled CNOT count, which changed both the reward and the evaluation. We introduced the matched-start comparison protocol after observing that naive method comparisons were not controlled. We scaled from H_2 to multiple LiH active spaces, which required GPU training and exposed the scalability limits reported above.

10 AI Tools Disclosure

We used AI assistants (Claude, ChatGPT, and GitHub Copilot) in a supporting role. They helped with peripheral and plumbing tasks: boilerplate and scaffolding, routine debugging, plotting and figure-generation scripts, and editing of this report for formatting, grammar, and passive voice. The core technical work was our own. We designed and implemented the reinforcement-learning components, including the PPO actor-critic, the environments, the reward design, the behavioral-cloning warm start, the curriculum, and the training and evaluation pipeline. We also implemented the quantum-chemistry components, including the VQE inner loop, ADAPT-VQE, the Hamiltonian and active-space construction, and the gate compilation, with AI assistance limited to debugging and small refactors rather than design. All experimental decisions, results, and analysis are our own.

References

- Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J Love, Alán Aspuru-Guzik, and Jeremy L O’Brien. A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5(1):4213, 2014.
- Jarrold R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics*, 18(2):023023, 2016.
- John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018.
- Abhinav Kandala, Antonio Mezzacapo, Kristan Temme, Maika Takita, Markus Brink, Jerry M Chow, and Jay M Gambetta. Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671):242–246, 2017.
- Harper R Grimsley, Sophia E Economou, Edwin Barnes, and Nicholas J Mayhall. An adaptive variational algorithm for exact molecular simulations on a quantum computer. *Nature Communications*, 10(1):3007, 2019.
- Ho Lun Tang, VO Shkolnikov, George S Barron, Harper R Grimsley, Nicholas J Mayhall, Edwin Barnes, and Sophia E Economou. Qubit-ADAPT-VQE: An adaptive algorithm for constructing hardware-efficient ansätze on a quantum processor. *PRX Quantum*, 2(2):020310, 2021.
- Mateusz Ostaszewski, Lea M Trenkwalder, Wojciech Masarczyk, Eleanor Scerri, and Vedran Dunjko. Reinforcement learning for optimization of variational quantum circuit architectures. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 18182–18194, 2021.
- Akash Kundu, Aritra Sarkar, and Abhishek Sadhu. Enhancing variational quantum state diagonalization using reinforcement learning techniques and curriculum-based learning. *arXiv preprint arXiv:2306.11086*, 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, Shah Nawaz Ahmed, Vishnu Ajith, M Sohaib Alam, Adrián Delgado, Soran Jahangiri, et al. PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv preprint arXiv:1811.04968*, 2018.

A Modal GPU campaign

The volume-verified GPU runs that underlie the LiH results are the three one-double seeds that match greedy at seven CNOTs, the two completed ADAPT-start seeds that reach twenty and eleven CNOTs against greedy’s twenty-three, and the six-qubit one-double seed that matches greedy at six CNOTs. These are the rows reported in Tables 3 and 4.

B Reproducibility

Headline numbers are aggregated in `results/final_report/master_results.json`, and the figures are regenerated by `consolidate_results.py` followed by `make_final_report_figures.py`. The matched-start campaign is driven by `lih_campaign.py` with shared settings in `lih_campaign_config.py`, and GPU jobs are dispatched by `modal_lih.py`. The H₂ and local LiH results reproduce on CPU after `pip install -r requirements.txt`. The GPU campaign numbers require a Modal account.